Initial Project Report - CSCE 2200 Team 7

A2 - Autocomplete + Ranked Search Engine

Problem Definition & Data:

The goal of our project is to develop an autocomplete and ranked search engine that suggests relevant words based on a prefix entered by the user. Autocomplete is usually used in search engines, text editors, and other applications to reduce typing effort and help users find words more efficiently. However, simply finding words that begin with a given prefix is not enough, because some suggestions are more relevant than others. We address this by using word frequencies to prioritize the most commonly occurring matches and return up to five ranked suggestions.

To achieve this, our project combines several data structures. A Trie supports efficient prefix-based searching, while a Max-Heap prioritizes matching words according to their frequencies. A Hash Table provides fast exact-word lookups, allowing the system to check whether a complete word exists in the dictionary. Additionally, a Circular Queue is used to maintain a limited history of recent searches. Together, these structures are intended to support autocomplete, ranking, word lookup, and search history within one application.

Our data consists of a dictionary containing words and their corresponding frequencies. Higher frequencies indicate a higher priority. After each dictionary entry is processed, the dataset is then loaded into memory so that the system can perform searches without repeatedly reading the dictionary file.

System Architecture and Component Design:

The architecture consists of a File Parser, Trie, Max-Heap, Hash Table, and Circular Queue. The intended workflow begins when the user enters a prefix. The prefix is then normalized and passed to the Trie, which retrieves up to five cached matching words. These

suggestions are then inserted into the Max-Heap and extracted according to their frequencies before being returned to the application for display. The Hash Table supports exact-word queries, while the Circular Queue supports recent-search history. The main application will coordinate these operations once the full integration is completed. Each component is explained fully below.

The File Parser is responsible for reading the dictionary and preparing its contents for storage. It processes each line, normalizes words to lowercase, validates their frequencies, and inserts valid entries into the Trie. It also tracks the number of successfully processed entries and skipped invalid lines. It connects the Trie and Max-Heap through the ranked-search function during autocomplete.

The Trie is the main data structure used for prefix searching. Each Trie node contains an array of 26 child pointers, representing the lowercase English alphabet. Words are stored along paths of connected nodes, and terminal nodes indicate complete words and store their frequencies. To improve autocomplete performance, each node also maintains a cache of up to five highest-frequency completions for its corresponding prefix. This allows the system to retrieve suggestions directly from the prefix node without traversing all its descendants during every search. The Trie also has insertion, exact-word searching, prefix checking, and deletion, with cached suggestions rebuilt after a word is removed.

The Max-Heap prioritizes autocomplete suggestions by frequency. It uses a dynamically allocated array to represent a binary heap, where the element with the highest frequency is always at the root. When a suggestion is inserted, the heap restores its ordering using a heapify-up operation. When the highest-priority suggestion is extracted, it restores the ordering using heapify-down. During ranked search, the cached suggestions retrieved from the Trie are inserted into the Max-Heap and then extracted in descending frequency order.

The Hash Table is for exact-word lookups. It uses a polynomial-style hash function with a multiplier of 31 to determine the bucket for each word. Collisions are handled through separate chaining, where linked-list nodes store words that map to the same bucket. The Hash Table has insertion, searching, and deletion, and automatically resizes when its load factor exceeds 75%. This helps limit collisions as more words are added.

The Circular Queue is designed to maintain a fixed-size history of recent searches. It stores strings in a dynamically allocated array and uses circular indexing to reuse available positions. When the queue becomes full, inserting a new search overwrites the oldest entry. This allows the system to retain recent activity without continuously increasing memory usage.

Design Trade-offs & Implementation Decisions:

In our project, we made certain design trade-offs and implementation choices to optimize efficiency and performance. Instead of relying on one data structure, we decided to implement both a Trie and a hash table. Although this uses extra memory, this also gives us better performance as the tree handles prefix autocomplete, while the hash table handles exact-word lookups. Another critical decision we made is loading the whole dictionary into the memory when the program starts, this uses more memory, but also avoids slow disk reads every time the user searches. Furthermore, our hash table automatically resizes at 75% full, which results in less crowded buckets and thus faster searches. Lastly, our Trie caches the top suggestions at every node, which prevents the need to scan the whole tree while the user types.

Planned Features & Extensions:

For milestone 2, one of our main goals will be to have our application run in real time instead of having to hit a specific key that triggers the application. Going beyond the core data structures of Trie, Max-Heap, Hash-Table and Circular Queue, the search engine may also incorporate advanced features, such as implementing a compressed Radix/Patricia tree data structure to cut down the memory usage when compared to a standard Trie. Another useful addition to our project could be a fuzzy-matching search that would catch typos by calculating edit distances (also called Levenshtein distance) and cutting off dead-end branches. As for the overall final submission goal for our project, we have two possible paths: we may try to publish this project as a browser extension, which would give us something that extends beyond the course and that we could publish online and be proud of. The other path involves us trying to tailor this project to the AUC organisation by adding AUC-specific terms, such as “SSE” and “Waleed”, to the dictionary and boosting the frequencies of words that carry a double/different

meaning in an AUC context, such as “declaration” and “core”, so that our project can potentially benefit the AUC community.

Division of Labor & Components (Milestone 1):

Component	Description	Responsible Member(s)
Trie Class	Core prefix tree structure supporting word insertion and cached autocomplete	Karimeddin Allam
Max-Heap Class	Binary heap sorting and prioritising autocomplete suggestions by frequency	Eyad Fouda
Hash Table Class	Separate chaining hash table providing fast exact word lookups.	Karimeddin Allam
File Parser Class	Input file parsing handling stream reading and token normalisation	Sara Eldababy
Circular Queue Class	Fixed size ring buffer for tracking recent search history	Yousef Ghobary
CMake Build System	Project configuration managing compilation targets and file paths	Yousef Ghobary
Application Driver (main.cpp)	Interactive execution loop that connects all project modules together	Yousef Ghanem
Initial Project Report	Documentation covering system architecture and design choices	Eyad Fouda / Sara Eldababy

GitHub Repository:

https://github.com/EyadFouda/ADS-Project-A2.git